

PROGRAMMAZIONE II

PACKAGE E VISIBILITÀ DELLE CLASSI

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

PACKAGE



MAGGIORE MODULARITÀ

Un **package** è un gruppo di classi, logicamente correlate

- L'incapsulamento aggiunge modularità al nostro codice, con i package scaliamo

Si pensi a un package come a una libreria, la cui interfaccia è l'insieme delle sue classi

- Tecnicamente, un package è una directory di file `.java`

num

Rational.java	Natural.java
Complex.java	Real.java

calc

Calculator.java
Main.java

MAGGIORE MODULARITÀ (CONT.)

I package aiutano a rendere il codice più strutturato, soprattutto in grandi progetti

Un package definisce un nuovo namespace e scope

- ▶ La visibilità è limitata dai package
- ▶ Lo stesso nome di classe può essere usato in package diversi

Un package è identificato da un nome gerarchico (**fully qualified name**)

- ▶ Ad esempio, `java.lang`

Convenzione di denominazione: nome Internet in ordine inverso

- ▶ Ad esempio, `it.univr.mypackage`

PACKAGE

I package possono essere annidati

`java.awt.event` è un sotto-package di `java.awt`

Per creare un package, aggiungere `package pkg;` all'inizio del file `.java`

- ▶ Se annidati, assicurarsi di creare l'albero delle directory

Per usare un package, usare la direttiva `import` nel file `.java` desiderato

- ▶ `import pkg.MyClass;` importerà una singola classe (`MyClass`) da `pkg`
- ▶ `import pkg.*;` importerà tutte le classi di `pkg` (senza sotto-package)

PACKAGE DI DEFAULT

Quando non è specificato alcun package, la classe appartiene al **package di default**

- ▶ Il package di default non ha nome
 - ▶ Le classi nel package di default non sono accessibili da classi di altri package
- ⚠ L'uso del package di default è una cattiva pratica ed è sconsigliato

ALBERO DELLE DIRECTORY

Punto di vista del compilatore (sorgente)

```
src/      non è un package
  num/
    Natural.java
    Rational.java
    Real.java
    Complex.java
  calc/
    Calculator.java
    Main.java
```

Punto di vista dell'interprete (bytecode)

```
bin/      non è un package
  num/
    Natural.class
    Rational.class
    Real.class
    Complex.class
  calc/
    Calculator.class
    Main.class
```

DEPLOYMENT CON I PACKAGE

```
package pkg;                                     // file sorgente: 1
public class MyClass {                           // src/pkg/MyClass.java 2
    public static void main(String[] args) {      3
        System.out.println("Hello_␣World!");     4
    }                                              5
}                                                  6
```

Compilare con `javac -d bin/ src/pkg/MyClass.java`

► Questo genererà `bin/pkg/MyClass.class`

Creare il package con `jar cvf MyJar.jar -C bin/ pkg/`

Eseguire con `java -cp MyJar.jar pkg.MyClass`

DEPLOYMENT CON I PACKAGE (CONT.)

Per definire una classe principale, aggiungere un file manifest

```
jar cvfm MyJar.jar manifest.txt -C bin/ pkg/
```

dove `manifest.txt` contiene la riga `Main-Class: pkg.MyClass`

Quindi, eseguire con `java -jar MyJar.jar`

-  In grandi progetti, lasciare che i build tool (es. Maven) gestiscano il packaging

BUILD E DEPLOYMENT AUTOMATICI

Usando uno strumento di build come Maven con un plugin adatto

```
<build>                                <!-- da aggiungere al file di configurazione pom.xml di Maven -->
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-assembly-plugin</artifactId>
      <version>3.7.1</version>
      <configuration>
        <descriptorRefs>
          <descriptorRef>jar-with-dependencies</descriptorRef>
        </descriptorRefs>
        <archive>
          <manifest>
            <addClasspath>true</addClasspath>
            <mainClass>pkg.MainClazz</mainClass>
          </manifest>
        </archive>
      </configuration>
    ...
```

BUILD E DEPLOYMENT AUTOMATICI (CONT.)

Usando uno strumento di build come Maven con un plugin adatto

```
...
    <executions>
      <execution>
        <id>assemble-all</id>
        <phase>package</phase>
        <goals>
          <goal>single</goal>
        </goals>
      </execution>
    </executions>
  </plugin>
</plugins>
</build>
```

Effettuare la build con `mvn package`

- Verrà creato un `projectName-1.0-SNAPSHOT-jar-with-dependencies.jar`

VISIBILITÀ DELLE CLASSI



RISOLUZIONE DEI PACKAGE

Ogni classe è locale al package in cui è definita

- ▶ Per usare una classe di un altro package bisogna indicare a Java dove trovarla

Con nomi fully qualified

```
package calc;

class Calculator {
    public static num.Real div(num.Natural a, num.Natural b) {
        // calcola la divisione di a per b
    }
}
```

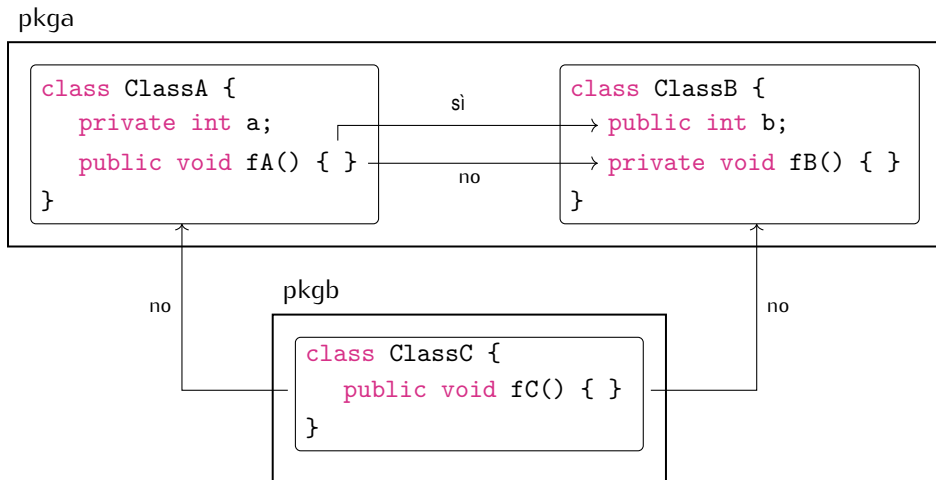
- Per usare una classe di un altro package

- Isando le direttive import.

```
import num.Real;      // questi due import possono essere sostituiti
import num.Natural;   // da un singolo import num.*

class Calculator {
    public static Real div(Natural a, Natural b) {
        // calcola la divisione di a per b
    }
}
```

VISIBILITÀ DELLE CLASSI



VISIBILITÀ DELLE CLASSI

pkgA

```
public class ClassA {  
    private int a;  
    public void fA() { }  
}
```

sì

no

```
class ClassB {  
    public int b;  
    private void fB() { }  
}
```

sì

pkgB

```
class ClassC {  
    public void fC() { }  
}
```

no



a



fA()

ACCESSO DI DEFAULT ALLE CLASSI

Quando omesso, l'accesso di default alla classe è `package-private`

- ▶ La classe è visibile solo da classi dello stesso package

In generale, le classi non possono essere definite come `private`

- ▶ La classe e i suoi membri sarebbero visibili solo a se stessi

REGOLE DI ACCESSO

Regole di accesso per i membri di una classe (campi e metodi)

	stessa classe	stesso package	altro package
membro private in qualsiasi classe	✓	✗	✗
membro public in classe (package)	✓	✓	✗
membro public in classe public	✓	✓	✓

... le cose si complicheranno a breve

DOMANDE E RISPOSTE

